

AI Engineer Internship Assignment

Build a Local-First Support Agent Network

Expected effort: 3–4 hours, excluding model download time

Interface: CLI, notebook or minimal API

Primary tools: Local Hugging Face models and LangGraph, or an equivalent graph-based orchestration framework

Objective

Build an AI agent network that answers support questions using a supplied fictional product knowledge base. The complete workflow must run locally after the required models have been downloaded. Remote language-model APIs must not be used.

Materials Provided

The assignment package will contain:

- 8–10 short Markdown knowledge-base documents
- A small JSON file containing previously resolved support cases
- Five sample user questions
- A starter output schema

The material concerns a fictional software product and requires no specialist domain knowledge.

Example Questions

- “My scheduled exports stopped after I changed my workspace timezone. What should I check?”
- “Can a read-only user create API credentials?”
- “The suggested solution did not work. What information should I collect before escalating?”
- “Write a refund for my subscription.” This request is intentionally outside the knowledge base and should be handled safely.

Technical Requirements

Build a graph-based workflow with the following four responsibilities. These may be implemented as individual nodes or as clearly separated groups of nodes.

1. Triage

Classify each incoming request as one of the following:

- Answerable from the knowledge base
- Requires clarification
- Requires escalation
- Out of scope

2. Retrieval

Retrieve relevant passages from the supplied documents or resolved cases. Use a locally running Hugging Face embedding, reranking or classification model. A managed vector database is not required.

3. Response Generation

Use a locally running Hugging Face language model to generate an answer based only on the retrieved evidence. Small CPU-compatible or quantized models are acceptable. Model quality is less important than correct integration and sound engineering judgment.

4. Verification

Check whether the generated response:

- Is supported by the retrieved evidence
- Contains source references
- Follows the required output schema
- Avoids inventing unsupported instructions

If verification fails, the graph should revise the answer once or return a safe failure response.

Orchestration Requirements

The graph must demonstrate:

- Shared typed state
- Conditional routing

- At least one retry, revision or fallback path
- Clear separation between deterministic code and model reasoning
- Logs showing which nodes executed
- Protection against an infinite graph loop

Simply calling multiple prompts sequentially without graph-based routing is not sufficient.

Local Model Requirements

- Use at least one model loaded through Hugging Face transformers, sentence-transformers or another Hugging Face library.
- Response generation must use a local model.
- State the exact model names and revisions used.
- Record approximate model load time and response latency.
- After initial setup, the application should be demonstrable with network access disabled.
- Do not use OpenAI, Anthropic, Gemini or other hosted language-model APIs.

Candidates may choose any suitable models. Model selection and hardware-aware trade-offs are part of the assignment.

Required Output

Return both a readable response and structured JSON in this general form:

```
{
  "classification": "answerable",
  "answer": "Suggested response",
  "sources": [
    {
      "document": "workspace-settings.md",
      "passage": "Relevant excerpt or passage identifier"
    }
  ],
  "confidence": 0.82,
  "requires_human": false,
  "reason": "Brief explanation"
}
```

You may improve or replace this schema.

Required Test Cases

Demonstrate the workflow on at least five cases:

1. A directly answerable question
2. A question requiring information from two documents
3. An ambiguous question requiring clarification
4. An out-of-scope request
5. A case where the initial generated answer fails verification

At least one automated test must verify graph routing without depending on the exact wording produced by the model.

Submission Through Google Form

Submit the following items through the Google Form shared with you:

- GitHub repository link containing the source code, README, setup instructions, test cases and sample outputs
- Graph diagram uploaded as a PNG or JPG image
- Video recording link
- Exact model names and revisions used
- Hardware requirements and the hardware used for your run, including CPU, RAM and GPU or accelerator if applicable

Ensure that all submitted links are accessible to reviewers without requesting additional permission.

Video Recording Requirements

Record a concise 4–7 minute walkthrough. The recording should show:

- A brief explanation of the graph and the responsibility of each node
- The local model being loaded, including the model name and device used
- At least three live runs that exercise different routes, including one answerable request and one clarification, escalation or out-of-scope request
- The execution logs or traces showing which nodes ran and which conditional path was selected
- The retrieved evidence and source references used for at least one answer

- The verification, retry or safe-failure path being triggered at least once
- A short explanation of one design trade-off, one known limitation and what you would improve with more time

The recording does not need professional editing. It should make the system's actual behaviour and your understanding of the implementation easy to verify.

Rules and Scope

- Spend no more than 3–4 hours on the assignment, excluding model download time.
- AI coding assistants are permitted and encouraged, but their use must be disclosed in the README.
- Remote language-model APIs are not permitted.
- Do not include API keys, credentials or other secrets in the repository.
- A sophisticated interface, deployment, authentication and production infrastructure are not expected.
- Copying an existing tutorial without meaningful modification is not acceptable.
- If you do not complete a requirement within the time limit, document what remains and explain the approach you would take.
- Correct orchestration, traceability and engineering judgment matter more than the writing quality of a small local model.